0146. LRU Cache

y: Medium | Category: Linked List | LeetCode: https://leetcode.com/problems/lru-

Problem

- Design a cache of fixed capacity with get(key) and put(key, value) in O(1). When full, evict the least recently used key.

Intuition

- HashMap gives O(1) lookup; a doubly linked list gives O(1) "move to front" and "remove from back".
- Map key → list node; the list is ordered from most to least recently used.

Brute Force

- Store timestamps and scan for the oldest on eviction: O(n) eviction.
- In Java, LinkedHashMap(accessOrder = true) with removeEldestEntry is a valid shortcut; interviewers usually want the manual version.

Optimal Approach

- Sentinel head and tail nodes.
- Get: if present, unlink the node and insert after head; return value.
- Put: if present, update and move to front; else create, insert at front, and if size > capacity remove tail.prev from list and map.

Worked Example

```
capacity 2.

put(1,1)  list: 1
put(2,2)  list: 2,1
get(1)=1  list: 1,2
put(3,3)  evict 2 -> list: 3,1
get(2)=-1
put(4,4)  evict 1 -> list: 4,3
get(1)=-1 get(3)=3 get(4)=4
```

Java Solution

```java
public class LRUCache {
    private static class Node {
        int key, val;
        Node prev, next;
        Node(int k, int v) { key = k; val = v; }
    }

    private final int capacity;
    private final Map<Integer, Node> map = new HashMap<>();
    private final Node head = new Node(0, 0), tail = new Node(0, 0);

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public int get(int key) {
        Node n = map.get(key);
        if (n == null) return -1;
        unlink(n);
        addFront(n);
        return n.val;
    }

    public void put(int key, int value) {
        Node n = map.get(key);
        if (n != null) {
            n.val = value;
            unlink(n);
            addFront(n);
            return;
        }
        n = new Node(key, value);
        map.put(key, n);
        addFront(n);
        if (map.size() > capacity) {
            Node lru = tail.prev;
            unlink(lru);
            map.remove(lru.key);
        }
```

Java Solution (2)

```
    }

    private void unlink(Node n) { n.prev.next = n.next; n.next.prev = n.prev; }

    private void addFront(Node n) {
        n.next = head.next;
        n.prev = head;
        head.next.prev = n;
        head.next = n;
    }
}
```

Complexity

- O(1) per operation. Space O(capacity).

Pitfalls

- A node must store its key so eviction can delete it from the map.
- Sentinels avoid all null checks.